컴퓨트 워커 (Compute Worker)

소개

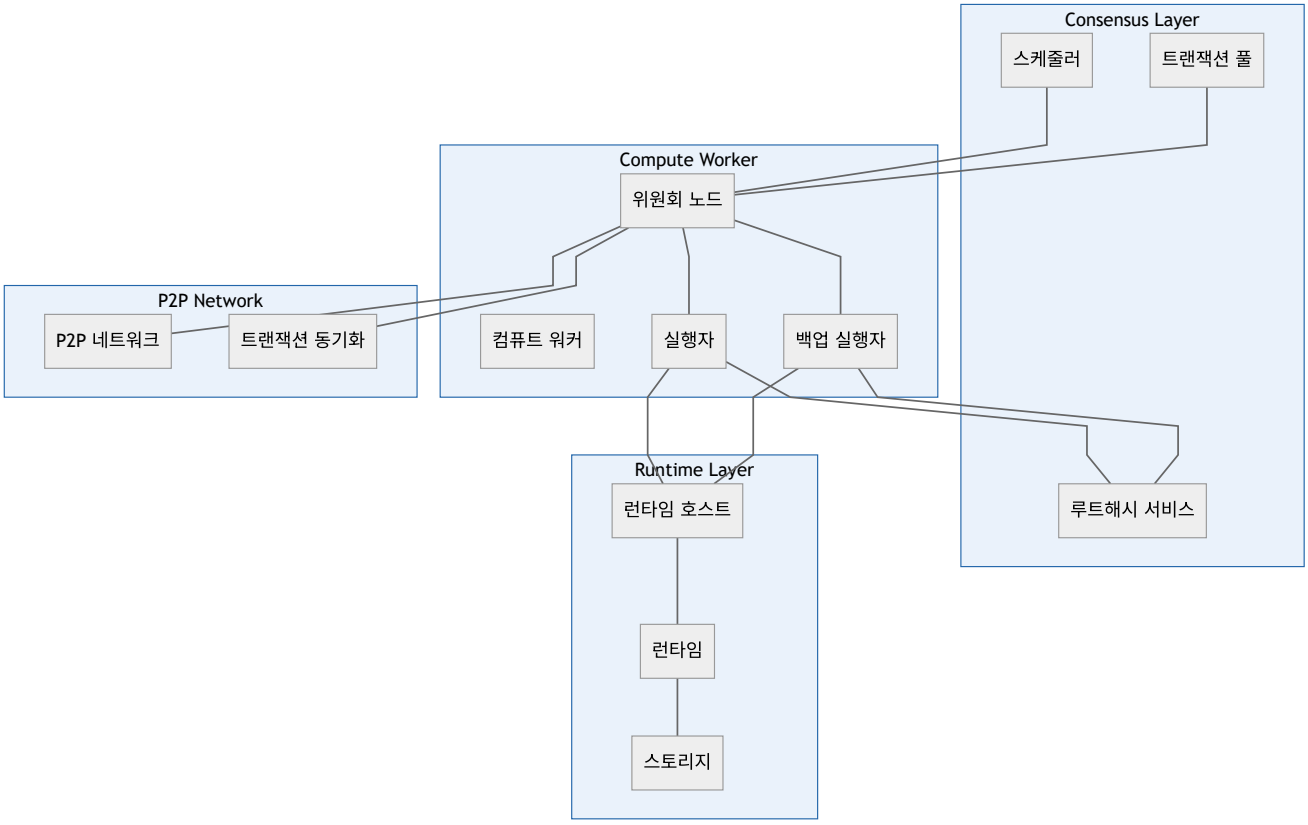
컴퓨트 워커는 Oasis Core의 런타임 계층에서 트랜잭션 실행을 담당하는 핵심 컴포넌트입니다. 컴퓨트 워커는 스케줄러로부터 트랜잭션 배치를 받아 런타임에서 실행하고, 실행 결과를 커밋하여 네트워크에 제출합니다.

컴퓨트 워커는 두 가지 주요 역할을 수행합니다:

- **실행자 워커 (Executor Worker)**: 트랜잭션 배치를 스케줄링하고 실행
- **백업 실행자 워커 (Backup Executor Worker)**: 불일치 탐지 시 검증 실행 수행

아키텍처

전체 구조



주요 컴포넌트

1. **컴퓨트 워커**: 여러 런타임을 관리하는 최상위 워커
2. **위원회 노드**: 개별 런타임의 위원회 참여를 담당
3. **실행자**: 트랜잭션 배치 스케줄링과 실행
4. **백업 실행자**: 불일치 탐지 시 검증 실행
5. **트랜잭션 동기화**: P2P를 통한 트랜잭션 동기화

컴퓨트 워커 구현

워커 구조체

```go
// Worker는 여러 런타임을 처리하는 실행자 워커입니다
type Worker struct {
enabled bool // 워커 활성화 여부
commonWorker *workerCommon.Worker // 공통 워커 서비스
registration *registration.Worker // 등록 서비스
runtimes map[common.Namespace]*committee.Node // 런타임별 위원회 노드
ctx context.Context // 컨텍스트
cancelCtx context.CancelFunc // 취소 함수
quitCh chan struct{} // 종료 채널
initCh chan struct{} // 초기화 채널
logger *logging.Logger // 로거
}
```

워커 생성 및 초기화

```go
// New는 새로운 실행자 워커를 생성합니다
func New(
commonWorker *workerCommon.Worker,
registration *registration.Worker,
) (*Worker, error) {
```

```
ctx, cancelCtx := context.WithCancel(context.Background())
// 컴퓨트 모드에서만 워커 활성화
var enabled bool
switch config.GlobalConfig.Mode {
case config.ModeCompute:
enabled = true
default:
enabled = false
}
w := &Worker{
enabled: enabled,
commonWorker: commonWorker,
registration: registration,
runtimes: make(map[common.Namespace]*committee.Node),
ctx: ctx,
cancelCtx: cancelCtx,
quitCh: make(chan struct{}),
initCh: make(chan struct{}),
logger: logging.GetLogger("worker/executor"),
}
if !enabled {
return w, nil
}
```

```go
// 설정된 모든 런타임 등록
for _, rt := range commonWorker.GetRuntimes() {
if err := w.registerRuntime(rt); err != nil {
return nil, fmt.Errorf("런타임 등록 실패: %w", err)
}
}
return w, nil
}

// registerRuntime은 새로운 런타임을 등록합니다
func (w *Worker) registerRuntime(commonNode *committeeCommon.Node) error {
id := commonNode.Runtime.ID()
w.logger.Info("새 런타임 등록",
"runtime_id", id,
)
// 역할 제공자 생성
rp, err := w.registration.NewRuntimeRoleProvider(node.RoleComputeWorker, id)
if err != nil {
return fmt.Errorf("역할 제공자 생성 실패: %w", err)
}
// 런타임용 위원회 노드 생성
node, err := committee.NewNode(
commonNode,
```

```go
w.commonWorker.GetConfig(),
rp,
)
if err != nil {
return fmt.Errorf("위원회 노드 생성 실패: %w", err)
}
commonNode.AddHooks(node)
w.runtimes[id] = node
w.logger.Info("새 런타임 등록 완료",
"runtime_id", id,
)
return nil
}
```

워커 생명주기 관리

```go
// Start는 워커를 시작합니다
func (w *Worker) Start() error {
if !w.enabled {
w.logger.Debug("워커가 비활성화되어 시작하지 않음")
close(w.initCh)
return nil
}
```

```go
// 모든 런타임과 프록시 종료 대기
go func() {
defer close(w.quitCh)
defer w.cancelCtx()
for _, rt := range w.runtimes {
<-rt.Quit()
}
}()
// 모든 런타임 초기화 및 노드 등록 대기
go func() {
for _, rt := range w.runtimes {
<-rt.Initialized()
}
<-w.registration.InitialRegistrationCh()
close(w.initCh)
}()
// 런타임 서비스 시작
for id, rt := range w.runtimes {
w.logger.Info("런타임 서비스 시작",
"runtime_id", id,
)
if err := rt.Start(); err != nil {
return fmt.Errorf("런타임 서비스 시작 실패: %w", err)
```

```go
}
}
return nil
}

// Stop은 워커를 중지합니다
func (w *Worker) Stop() {
if !w.enabled {
close(w.quitCh)
return
}
for id, rt := range w.runtimes {
w.logger.Info("런타임 서비스 중지",
"runtime_id", id,
)
rt.Stop()
}
}

// Cleanup은 워커 정리를 수행합니다
func (w *Worker) Cleanup() {
if !w.enabled {
return
```

```
}
for _, rt := range w.runtimes {
rt.Cleanup()
}
}
```

위원회 노드 구현

노드 상태 관리

```
// NodeState는 위원회 노드의 상태를 나타냅니다
type NodeState interface {
Name() string
}

// 가능한 노드 상태들
type (
StateWaitingForBatch struct{} // 배치 대기
StateWaitingForTxs struct { // 트랜잭션 대기
proposal *commitment.Proposal
rank uint64
batch transaction.RawBatch
txs map[hash.Hash]int
```

```go
bytes uint64
maxBytes uint64
batchSize uint64
maxBatchSize uint64
cancelFn context.CancelFunc
done chan struct{}
}
StateWaitingForEvent struct { // 이벤트 대기
proposal *commitment.Proposal
rank uint64
batch transaction.RawBatch
}
StateProcessingBatch struct { // 배치 처리 중
mode protocol.ExecutionMode
rank uint64
batchStartTime time.Time
cancelFn context.CancelCauseFunc
done chan struct{}
}
)

// 상태 전환 규칙
var validStateTransitions = map[string][]string{
```

```
"WaitingForBatch": {"WaitingForTxs", "WaitingForEvent", "ProcessingBatch"},
"WaitingForTxs": {"WaitingForBatch", "WaitingForEvent", "ProcessingBatch"},
"WaitingForEvent": {"WaitingForBatch", "ProcessingBatch"},
"ProcessingBatch": {"WaitingForBatch"},
}
```

배치 스케줄링

```
// scheduleBatch는 새로운 배치를 스케줄링합니다
func (n *Node) scheduleBatch(ctx context.Context, round  uint64, force  bool) {
n.logger.Debug("배치 스케줄링 시도",
"round", round,
"rank", n.rank,
"force", force,
"state", n.state.Name(),
)
// 배치 스케줄링에 적합한 상태인지 확인
switch  n.state.(type) {
case StateWaitingForBatch:
default:
return
}
// 한 번만 스케줄링
```

```
if  _, ok := n.submitted[n.rank];  ok {
n.logger.Debug("스케줄링 안함, 커밋이 이미 제출됨")
return
}
// 실행자 워커만 배치 스케줄링 가능
if !n.epoch.IsExecutorWorker() {
n.logger.Debug("스케줄링 안함, 실행자가 아님")
return
}
// 다음 블록이 에포크 전환 블록인 경우 제안하지 않음
epochState, err := n.commonNode.Consensus.Beacon().GetFutureEpoch(ctx, n.blockIn
if  err != nil {
n.logger.Error("미래 에포크 상태 가져오기 실패", "err", err)
return
}
if  epochState != nil && epochState.Height == n.blockInfo.ConsensusBlock.Height+
n.logger.Debug("스케줄링 안함, 다음 합의 블록이 에포크 전환")
return
}
// 들어오는 메시지 큐 메타데이터 가져오기
inMsgMeta, err := n.commonNode.Consensus.RootHash().GetIncomingMessageQueueMeta(
RuntimeID: n.commonNode.Runtime.ID(),
Height: consensus.HeightLatest,
```

```
})
if  err != nil {
n.logger.Error("들어오는 런타임 메시지 큐 메타데이터 가져오기 실패", "err", err)
return
}
// 런타임이 지원하는 기능 확인
rtInfo, err := n.rt.GetInfo(ctx)
if  err != nil {
n.logger.Warn("스케줄링 안함, 런타임이 손상됨", "err", err)
return
}
if !rtInfo.Features.HasScheduleControl() {
n.logger.Error("런타임이 스케줄 제어를 지원하지 않음")
return
}
// 트랜잭션 풀에서 배치 가져오기
batch := n.commonNode.TxPool.GetSchedulingSuggestion(rtInfo.Features.ScheduleCon
// 배치 스케줄링 조건 확인
switch {
case  force:
// 배치 플러시 타임아웃 만료, 빈 배치 스케줄링
case  len(batch) > 0:
// 트랜잭션이 있음, 배치 스케줄링
```

```
case  len(n.roundResults.Messages) > 0:
// 런타임 메시지 결과가 있음, 배치 스케줄링
case  inMsgMeta.Size > 0:
// 대기 중인 들어오는 런타임 메시지가 있음, 배치 스케줄링
case  n.rtState.LastNormalRound == n.rtState.GenesisBlock.Header.Round:
// 런타임 제네시스, 배치 스케줄링
case  n.rtState.LastNormalHeight < n.epoch.GetEpochHeight():
// 이 에포크에서 런타임이 처리한 블록이 없음, 배치 스케줄링
default:
// 배치 스케줄링 불필요
n.logger.Debug("스케줄링 안함, 트랜잭션 없음")
n.commonNode.TxPool.FinishScheduling()
return
}
// 배치 처리 상태로 전환
ctx, cancel := context.WithCancelCause(ctx)
done := make(chan  struct{})
n.transitionState(StateProcessingBatch{
mode: protocol.ExecutionModeSchedule,
rank: n.rank,
batchStartTime: time.Now(),
cancelFn: cancel,
done: done,
```

```go
})

// 별도 고루틴에서 배치 스케줄링 시작

go func() {

defer close(done)

n.startSchedulingBatch(ctx, batch)

n.commonNode.TxPool.FinishScheduling()

}()

}
```

배치 실행

```go
// startSchedulingBatch는 배치 스케줄링을 시작합니다

func (n *Node) startSchedulingBatch(ctx context.Context, batch []*txpool.TxQueue

n.logger.Debug("배치 스케줄링",

"batch_size", len(batch),

)

// 초기 배치 준비

initialBatch := make([][]byte, 0, len(batch))

for _, tx := range batch {

initialBatch = append(initialBatch, tx.Raw())

}

// 런타임에 배치 실행 요청
```

```
rsp, err := n.runtimeExecuteTxBatch(
ctx,
n.rt,
protocol.ExecutionModeSchedule,
n.blockInfo.Epoch,
n.blockInfo.ConsensusBlock,
n.blockInfo.RuntimeBlock,
n.rtState,
n.roundResults,
hash.Hash{}, // IORoot는 아직 결정되지 않았으므로 무시
initialBatch,
)
if  err != nil {
n.logger.Error("런타임 배치 실행 실패", "err", err)
n.processedBatchCh <- nil
return
}
// 거부된 트랜잭션 제거
n.commonNode.TxPool.RejectTxs(rsp.TxRejectHashes)
// 제안된 트랜잭션 표시
_, _ = n.commonNode.TxPool.PromoteProposedBatch(rsp.TxHashes)
// 새 제안 생성
proposal := commitment.Proposal{
```

```
NodeID: n.commonNode.Identity.NodeSigner.Public(),
Header: commitment.ProposalHeader{
Round: n.blockInfo.RuntimeBlock.Header.Round + 1,
PreviousHash: n.blockInfo.RuntimeBlock.Header.EncodedHash(),
BatchHash: rsp.TxInputRoot,
},
Batch: rsp.TxHashes,
}
// 라운드 워커에 응답 제출
n.processedBatchCh <- &processedBatch{
proposal: &proposal,
rank: n.rank,
computed: &rsp.Batch,
txInputWriteLog: rsp.TxInputWriteLog,
}
}


// runtimeExecuteTxBatch는 런타임에서 트랜잭션 배치를 실행합니다
func (n *Node) runtimeExecuteTxBatch(
ctx context.Context,
rt host.RichRuntime,
mode protocol.ExecutionMode,
epoch beacon.EpochTime,
```

```
consensusBlk *consensus.LightBlock,
blk *block.Block,
state *roothash.RuntimeState,
roundResults *roothash.RoundResults,
inputRoot hash.Hash,
inputs transaction.RawBatch,
) (*protocol.RuntimeExecuteTxBatchResponse, error) {
// 블록 라운드가 스토리지에 동기화되었는지 확인
n.logger.Debug("블록 라운드 동기화 확인", "round", blk.Header.Round)
if _, err := n.commonNode.Runtime.History().WaitRoundSynced(ctx, blk.Header.Rou
return nil, fmt.Errorf("블록 라운드 동기화 대기 실패: %w", err)
}
// 들어오는 메시지 가져오기
inMsgs, err := n.commonNode.Consensus.RootHash().GetIncomingMessageQueue(ctx, &r
RuntimeID: n.commonNode.Runtime.ID(),
Height: consensusBlk.Height,
})
if err != nil {
n.logger.Error("들어오는 런타임 메시지 큐 가져오기 실패", "err", err)
return nil, fmt.Errorf("들어오는 메시지 가져오기 실패: %w", err)
}
// 런타임 실행 요청 준비
rq := &protocol.Body{
```

```
RuntimeExecuteTxBatchRequest: &protocol.RuntimeExecuteTxBatchRequest{

Mode: mode,

ConsensusBlock: *consensusBlk,

RoundResults: roundResults,

IORoot: inputRoot,

Inputs: inputs,

InMessages: inMsgs,

Block: *blk,

Epoch: epoch,

MaxMessages: state.Runtime.Executor.MaxMessages,

},

}

// 메트릭 기록

batchSize.With(n.getMetricLabels()).Observe(float64(len(inputs)))

rtStartTime := time.Now()

defer  func() {

batchRuntimeProcessingTime.With(n.getMetricLabels()).Observe(time.Since(rtStartT

}()

// 배치 실행 시간 제한

proposerTimeout := state.Runtime.TxnScheduler.ProposerTimeout

callCtx, cancelCallFn := context.WithTimeoutCause(

context.TODO(), // 런타임이 중단 요청을 지원하기 시작하면 ctx로 교체

executeBatchTimeoutFactor*proposerTimeout,
```

```
errors.New("제안자 타임아웃 만료"),
)
defer cancelCallFn()
// 런타임 호출
rsp, err := rt.Call(callCtx, rq)
switch {
case err == nil:
// 성공
case errors.Is(err, context.Canceled), errors.Is(err, context.DeadlineExceeded)
// 런타임이 요청을 처리하는 동안 컨텍스트가 취소됨
n.logger.Error("배치 처리가 컨텍스트에 의해 중단됨, 런타임 재시작",
"cause", context.Cause(callCtx),
)
// 다음 배치를 처리할 수 있도록 런타임 중단
abortCtx, cancel := context.WithTimeout(n.ctx, abortTimeout)
defer cancel()
if err = rt.Abort(abortCtx, false); err != nil {
n.logger.Error("런타임 중단 실패", "err", err)
}
return nil, fmt.Errorf("배치 처리가 컨텍스트에 의해 중단됨")
default:
n.logger.Error("런타임에 배치 처리 요청을 보내는 중 오류",
"err", err,
```

```
)
return  nil, fmt.Errorf("런타임 호출 실패: %w", err)
}
if  rsp.RuntimeExecuteTxBatchResponse == nil {
n.logger.Error("런타임에서 잘못된 응답",
"response", rsp,
)
return  nil, fmt.Errorf("런타임에서 잘못된 응답")
}
return  rsp.RuntimeExecuteTxBatchResponse, nil
}
```

트랜잭션 처리 흐름

1. 배치 제안 수신

```
// processProposal은 제안을 처리합니다
func (n *Node) processProposal(ctx context.Context, proposal *commitment.Proposa
n.logger.Debug("제안 처리 시도",
"scheduler", proposal.NodeID,
"round", proposal.Header.Round,
"rank", rank,
"discrepancy", discrepancy,
```

```
)
// 제안 처리에 적합한 상태인지 확인
switch  n.state.(type) {
case StateWaitingForBatch:
default:
n.logger.Debug("처리 안함, 잘못된 상태", "state", n.state.Name())
return
}
// 한 번만 처리
if  _, ok := n.submitted[rank];  ok {
n.logger.Debug("처리 안함, 커밋이 이미 제출됨")
return
}
// 권한 확인
switch  discrepancy {
case true:
// 백업 실행자 워커만 배치 처리 가능
if !n.epoch.IsExecutorBackupWorker() {
n.logger.Debug("처리 안함, 백업 실행자가 아님")
return
}
case false:
// 모든 워커가 배치 처리 가능
```

```
// 백업 실행자 워커는 불일치 이벤트를 기다림

}

n.logger.Debug("배치 해결 시도")

// 배치 해결 시도

resolvedBatch, missingTxs := n.commonNode.TxPool.PromoteProposedBatch(proposal.B

// 배치 크기 계산

batchSize := uint64(len(proposal.Batch))

bytes := uint64(0)

for  _, tx := range  resolvedBatch {

if  tx == nil {

continue

}

bytes += uint64(tx.Size())

}

// 배치가 유효하지 않으면 실패 제출

maxBatchSize := n.blockInfo.ActiveDescriptor.TxnScheduler.MaxBatchSize

maxBytes := n.blockInfo.ActiveDescriptor.TxnScheduler.MaxBatchSizeBytes

if  batchSize > maxBatchSize || maxBytes > 0 && bytes > maxBytes {

n.transitionStateToProcessingFailure(proposal, rank, bytes, maxBytes, batchSize,

return

}

// 배치 준비

batch := make(transaction.RawBatch, 0, len(resolvedBatch))
```

```go
for _, tx := range resolvedBatch {
switch tx {
case nil:
batch = append(batch, nil)
default:
batch = append(batch, tx.Raw())
}
}
// 누락된 트랜잭션이 있으면 대기
if len(missingTxs) > 0 {
n.logger.Debug("일부 트랜잭션이 누락됨", "num_missing", len(missingTxs))
txHashes := maps.Keys(missingTxs)
subCtx, cancelFn := context.WithCancel(ctx)
done := make(chan struct{})
n.transitionState(StateWaitingForTxs{
proposal: proposal,
rank: rank,
batch: batch,
txs: missingTxs,
bytes: bytes,
maxBytes: maxBytes,
batchSize: batchSize,
maxBatchSize: maxBatchSize,
```

```
cancelFn: cancelFn,
done: done,
})
go  func() {
defer  close(done)
n.requestMissingTransactions(subCtx, txHashes)
}()
return
}
// 올바른 블록이 있는지 확인
currentHash := n.blockInfo.RuntimeBlock.Header.EncodedHash()
if !currentHash.Equal(&proposal.Header.PreviousHash) {
return
}
// 백업 워커는 불일치가 탐지된 경우에만 처리
if !n.epoch.IsExecutorWorker() && n.epoch.IsExecutorBackupWorker() && !discrepan
n.transitionState(StateWaitingForEvent{
proposal: proposal,
rank: rank,
batch: batch,
})
return
}
```

```
n.transitionStateToProcessing(ctx, proposal, rank, batch)
}
```

2. 커밋 생성 및 제출

```
// proposeBatch는 배치를 제안합니다
func (n *Node) proposeBatch(
roundCtx context.Context,
lastHeader *block.Header,
processed *processedBatch,
) {
batch := processed.computed
n.logger.Debug("배치 제안",
"scheduler_id", processed.proposal.NodeID,
"node_id", n.commonNode.Identity.NodeSigner.Public(),
"batch_size", len(processed.proposal.Batch),
"io_root", *batch.Header.IORoot,
"state_root", *batch.Header.StateRoot,
"messages_hash", *batch.Header.MessagesHash,
"in_msgs_hash", *batch.Header.InMessagesHash,
"in_msgs_count", batch.Header.InMessagesCount,
)
// 실행자 커밋 생성
```

```
rakSig := batch.RakSig

ec := &commitment.ExecutorCommitment{

NodeID: n.commonNode.Identity.NodeSigner.Public(),

Header: commitment.ExecutorCommitmentHeader{

SchedulerID: processed.proposal.NodeID,

Header: batch.Header,

RAKSignature: &rakSig,

},

}

// 트랜잭션 스케줄러인 경우 모든 방출된 메시지 포함

if  ec.NodeID.Equal(ec.Header.SchedulerID) {

ec.Messages = batch.Messages

}

// I/O 및 상태 쓰기 로그를 스토리지에 커밋

storageErr := func() error {

start := time.Now()

defer  func() {

storageCommitLatency.With(n.getMetricLabels()).Observe(time.Since(start).Seconds

}()

ctx, cancel := context.WithCancel(roundCtx)

defer  cancel()

// 최종 I/O 루트 저장

var  emptyRoot hash.Hash
```

```
emptyRoot.Empty()
err := n.storage.Apply(ctx, &storage.ApplyRequest{
Namespace: lastHeader.Namespace,
RootType: storage.RootTypeIO,
SrcRound: lastHeader.Round + 1,
SrcRoot: emptyRoot,
DstRound: lastHeader.Round + 1,
DstRoot: *batch.Header.IORoot,
WriteLog: append(processed.txInputWriteLog, batch.IOWriteLog...),
})
if  err != nil {
return  fmt.Errorf("I/O 루트 적용 실패: %w", err)
}
// 상태 루트 업데이트
err = n.storage.Apply(ctx, &storage.ApplyRequest{
Namespace: lastHeader.Namespace,
RootType: storage.RootTypeState,
SrcRound: lastHeader.Round,
SrcRoot: lastHeader.StateRoot,
DstRound: lastHeader.Round + 1,
DstRoot: *batch.Header.StateRoot,
WriteLog: batch.StateWriteLog,
})
```

```
if err != nil {
return fmt.Errorf("상태 루트 적용 실패: %w", err)
}
// 커밋에 서명하기 전에 스토리지가 디스크에 쓰여졌는지 확인
return n.storage.NodeDB().Sync()
}()
if storageErr != nil {
n.logger.Error("스토리지 실패, 실패를 나타내는 커밋 제출",
"err", storageErr,
)
ec.Header.SetFailure(commitment.FailureUnknown)
}
// 커밋 제출
if err := n.signAndSubmitCommitment(roundCtx, ec); err != nil {
n.logger.Error("커밋 서명 및 제출 실패",
"commit", ec,
"err", err,
)
return
}
n.submitted[processed.rank] = struct{}{}
if storageErr != nil {
n.transitionState(StateWaitingForBatch{})
```

```
return
}
n.proposedBatch = &proposedBatch{
batchStartTime: state.batchStartTime,
proposedIORoot: *ec.Header.Header.IORoot,
txHashes: processed.proposal.Batch,
}
n.transitionState(StateWaitingForBatch{})
}


// signAndSubmitCommitment는 커밋에 서명하고 제출합니다
func (n *Node) signAndSubmitCommitment(roundCtx context.Context, ec *commitment.
// 커밋에 서명
err := ec.Sign(n.commonNode.Identity.NodeSigner, n.commonNode.Runtime.ID())
if err != nil {
return fmt.Errorf("커밋 서명 실패: %w", err)
}
// 트랜잭션 생성 및 제출
tx := roothash.NewExecutorCommitTx(0, nil, n.commonNode.Runtime.ID(), []commitme
go func() {
commitErr := consensus.SignAndSubmitTx(roundCtx, n.commonNode.Consensus, n.commo
switch commitErr {
case nil:
```

```
n.logger.Info("실행자 커밋 완료")
default:
n.logger.Error("실행자 커밋 제출 실패",
"commit", ec,
"err", commitErr,
)
}
}()
return  nil
}
```

불일치 탐지 및 처리

불일치 이벤트 처리

```
// handleDiscrepancyEvent는 불일치 이벤트를 처리합니다
func (n *Node) handleDiscrepancyEvent(ev *roothash.Event) {
if  ev.ExecutionDiscrepancyDetected == nil {
return
}
n.logger.Warn("실행 불일치 탐지됨",
"round", ev.ExecutionDiscrepancyDetected.Round,
)
```

```
// 불일치 이벤트 저장
n.discrepancy = &discrepancyEvent{
round: ev.ExecutionDiscrepancyDetected.Round,
}
// 백업 실행자인 경우 처리 시작
if  n.epoch.IsExecutorBackupWorker() {
n.logger.Info("백업 실행자로서 불일치 해결 시작")
// 현재 상태에 따라 처리
switch  state := n.state.(type) {
case StateWaitingForEvent:
// 대기 중인 이벤트가 있으면 처리 시작
n.transitionStateToProcessing(n.ctx, state.proposal, state.rank, state.batch)
default:
// 다른 상태에서는 상태 업데이트만 수행
n.updateState(n.ctx, 0, math.MaxUint64, true)
}
}
}
```

백업 실행자 동작

```
// 백업 실행자는 다음과 같은 특별한 동작을 수행합니다:
// 1. 일반적으로는 대기 상태 유지
```

```
// 2. 불일치 탐지 시에만 배치 실행
// 3. 실행 결과를 통해 올바른 결과 제공


func (n *Node) handleBackupExecution(ctx context.Context, proposal *commitment.P
n.logger.Info("백업 실행자로서 배치 실행 시작",
"round", proposal.Header.Round,
"rank", rank,
)
// 배치 실행
rsp, err := n.runtimeExecuteTxBatch(
ctx,
n.rt,
protocol.ExecutionModeExecute,
n.blockInfo.Epoch,
n.blockInfo.ConsensusBlock,
n.blockInfo.RuntimeBlock,
n.rtState,
n.roundResults,
proposal.Header.BatchHash,
batch,
)
if  err != nil {
n.logger.Error("백업 실행자 배치 실행 실패", "err", err)
```

```
return
}
// 실행 결과 검증
if  err := n.validateExecutionResult(rsp, proposal);  err != nil {
n.logger.Error("실행 결과 검증 실패", "err", err)
return
}
// 올바른 커밋 생성 및 제출
n.processedBatchCh <- &processedBatch{
proposal: proposal,
rank: rank,
computed: &rsp.Batch,
txInputWriteLog: rsp.TxInputWriteLog,
}
n.logger.Info("백업 실행자 배치 실행 완료")
}
```

P2P 통신 및 동기화

트랜잭션 동기화

```
// requestMissingTransactions는 누락된 트랜잭션을 요청합니다
func (n *Node) requestMissingTransactions(ctx context.Context, txHashes []hash.H
```

```
n.logger.Debug("누락된 트랜잭션 요청",
"num_missing", len(txHashes),
)
// P2P를 통해 트랜잭션 동기화 요청
if err := n.txSync.RequestTransactions(ctx, txHashes); err != nil {
n.logger.Error("트랜잭션 동기화 요청 실패",
"err", err,
"tx_hashes", txHashes,
)
return
}
// 트랜잭션 수신 대기
timeout := time.NewTimer(30 * time.Second)
defer timeout.Stop()
for len(txHashes) > 0 {
select {
case <-ctx.Done():
n.logger.Debug("트랜잭션 동기화 취소됨")
return
case <-timeout.C:
n.logger.Warn("트랜잭션 동기화 타임아웃",
"remaining", len(txHashes),
)
```

```
return
case  txs := <-n.missingTxCh:
// 수신된 트랜잭션 처리
for  _, txData := range  txs {
txHash := hash.NewFromBytes(txData)
// 요청한 트랜잭션인지 확인
for  i, reqHash := range  txHashes {
if  txHash.Equal(&reqHash) {
// 트랜잭션 풀에 추가
if  err := n.commonNode.TxPool.AddTransaction(txData);  err != nil {
n.logger.Error("트랜잭션 추가 실패",
"err", err,
"tx_hash", txHash,
)
continue
}
// 목록에서 제거
txHashes = append(txHashes[:i], txHashes[i+1:]...)
break
}
}
}
}
```

```
}
n.logger.Debug("모든 누락된 트랜잭션 수신 완료")
}
```

제안 전파

```
// publishProposal은 제안을 P2P 네트워크에 전파합니다
func (n *Node) publishProposal(ctx context.Context, proposal *commitment.Proposa
// 제안에 서명
if  err := proposal.Sign(n.commonNode.Identity.NodeSigner, n.commonNode.Runtime.
return  fmt.Errorf("제안 서명 실패: %w", err)
}
n.logger.Debug("새 배치 제안 전파",
"input_root", proposal.Header.BatchHash,
"batch_size", len(proposal.Batch),
)
// P2P 네트워크에 제안 전파
n.commonNode.P2P.Publish(ctx, n.committeeTopic, &p2p.CommitteeMessage{
Epoch: n.blockInfo.Epoch,
Proposal: proposal,
})
return  nil
```

```
}
```

성능 최적화

배치 크기 최적화

```
// optimizeBatchSize는 네트워크 상황에 따라 배치 크기를 최적화합니다
func (n *Node) optimizeBatchSize(rtInfo *protocol.RuntimeInfoResponse) uint32 {
baseSize := rtInfo.Features.ScheduleControl.InitialBatchSize
// 네트워크 지연 시간 고려
avgLatency := n.getAverageNetworkLatency()
if avgLatency > 500*time.Millisecond {
// 높은 지연 시간에서는 배치 크기 증가
return min(baseSize*2, rtInfo.Features.ScheduleControl.MaxBatchSize)
}
// 트랜잭션 풀 크기 고려
poolSize := n.commonNode.TxPool.Size()
if poolSize > 1000 {
// 많은 대기 트랜잭션이 있으면 배치 크기 증가
return min(baseSize*3/2, rtInfo.Features.ScheduleControl.MaxBatchSize)
}
return baseSize
}
```

```go
// getAverageNetworkLatency는 평균 네트워크 지연 시간을 계산합니다
func (n *Node) getAverageNetworkLatency() time.Duration {
// P2P 피어들과의 평균 지연 시간 계산
peers := n.commonNode.P2P.GetPeers()
if len(peers) == 0 {
return 100 * time.Millisecond // 기본값
}
var totalLatency time.Duration
validPeers := 0
for _, peer := range peers {
if latency := peer.GetLatency(); latency > 0 {
totalLatency += latency
validPeers++
}
}
if validPeers == 0 {
return 100 * time.Millisecond // 기본값
}
return totalLatency / time.Duration(validPeers)
}
```

메모리 관리

```go
// 메모리 사용량 최적화를 위한 배치 처리
type batchProcessor struct {
maxMemoryUsage int64
currentUsage int64
mu sync.RWMutex
}

func (bp *batchProcessor) canProcessBatch(batchSize int64) bool {
bp.mu.RLock()
defer bp.mu.RUnlock()
return bp.currentUsage+batchSize <= bp.maxMemoryUsage
}

func (bp *batchProcessor) allocateBatch(batchSize int64) bool {
bp.mu.Lock()
defer bp.mu.Unlock()
if bp.currentUsage+batchSize > bp.maxMemoryUsage {
return false
}
bp.currentUsage += batchSize
return true
}
```

```go
func (bp *batchProcessor) releaseBatch(batchSize int64) {
bp.mu.Lock()
defer bp.mu.Unlock()
bp.currentUsage -= batchSize
if bp.currentUsage < 0 {
bp.currentUsage = 0
}
}
```

모니터링 및 메트릭

성능 메트릭

```go
var (
// 배치 처리 메트릭
batchSize = prometheus.NewHistogramVec(
prometheus.HistogramOpts{
Name: "oasis_worker_batch_size",
Help: "배치 크기 분포",
},
[]string{"runtime"},
)
```

```
batchRuntimeProcessingTime = prometheus.NewHistogramVec(
prometheus.HistogramOpts{
Name: "oasis_worker_batch_runtime_processing_time",
Help: "런타임 배치 처리 시간",
},
[]string{"runtime"},
)
storageCommitLatency = prometheus.NewHistogramVec(
prometheus.HistogramOpts{
Name: "oasis_worker_storage_commit_latency",
Help: "스토리지 커밋 지연 시간",
},
[]string{"runtime"},
)
abortedBatchCount = prometheus.NewCounterVec(
prometheus.CounterOpts{
Name: "oasis_worker_aborted_batch_total",
Help: "중단된 배치 총 수",
},
[]string{"runtime"},
)
)
```

```go
func (n *Node) getMetricLabels() prometheus.Labels {
return prometheus.Labels{
"runtime": n.commonNode.Runtime.ID().String(),
}
}
```

상태 모니터링

```go
// GetStatus는 컴퓨트 워커의 현재 상태를 반환합니다
func (w *Worker) GetStatus() *WorkerStatus {
if !w.enabled {
return &WorkerStatus{
Enabled: false,
}
}
runtimeStatuses := make(map[string]*RuntimeStatus)
for  id, rt := range  w.runtimes {
runtimeStatuses[id.String()] = &RuntimeStatus{
ID: id,
State: rt.GetCurrentState().Name(),
Epoch: rt.GetCurrentEpoch(),
Round: rt.GetCurrentRound(),
}
```

```
}
return &WorkerStatus{
Enabled: true,
Runtimes: runtimeStatuses,
}
}

type WorkerStatus struct {
Enabled  bool  `json:"enabled"`
Runtimes  map[string]*RuntimeStatus `json:"runtimes,omitempty"`
}

type RuntimeStatus struct {
ID common.Namespace `json:"id"`
State  string  `json:"state"`
Epoch  uint64  `json:"epoch"`
Round  uint64  `json:"round"`
}
```

설정 및 사용 예제

컴퓨트 워커 설정

```yaml
# oasis-node 설정 파일
mode: compute

worker:
compute:
enabled: true
# 런타임별 설정
runtimes:
- id: "8000000000000000000000000000000000000000000000000000000000000000"
bundle_path: "/opt/oasis/runtime.orc"
runtime:
# 런타임 레지스트리 설정
registries:
- "https://registry.oasisprotocol.org"
# 번들 크기 제한
max_bundle_size: "32MB"
# SGX 설정 (필요한 경우)
sgx:
loader: "/opt/oasis/oasis-core-runtime-loader"

# P2P 네트워크 설정
p2p:
```

```yaml
enabled: true

port: 9200

# 합의 설정

consensus:

backend: tendermint

tendermint:

core:

listen_address: "tcp://0.0.0.0:26656"
```

컴퓨트 워커 실행

```bash
# 컴퓨트 워커 노드 실행

oasis-node \

--config /etc/oasis/node.yml \

--datadir  /opt/oasis/node \

--log.level info \

--log.format  json


# 워커 상태 확인

oasis-node  control  status


# 런타임 상태 확인

oasis-node  control  runtime-stats
```

```
# 메트릭 확인
curl  http://localhost:9001/metrics | grep  oasis_worker
```

프로그래밍 방식 사용

```
package main

import (
"context"
"fmt"
"github.com/oasisprotocol/oasis-core/go/worker/compute/executor"
"github.com/oasisprotocol/oasis-core/go/worker/common"
"github.com/oasisprotocol/oasis-core/go/worker/registration"
)

func  main() {
// 공통 워커 생성
commonWorker, err := common.New(/* 설정 */)
if  err != nil {
panic(fmt.Sprintf("공통 워커 생성 실패: %v", err))
}
// 등록 워커 생성
```

```go
registrationWorker, err := registration.New(/* 설정 */)
if err != nil {
panic(fmt.Sprintf("등록 워커 생성 실패: %v", err))
}
// 컴퓨트 워커 생성
computeWorker, err := executor.New(commonWorker, registrationWorker)
if err != nil {
panic(fmt.Sprintf("컴퓨트 워커 생성 실패: %v", err))
}
// 워커 시작
if err := computeWorker.Start(); err != nil {
panic(fmt.Sprintf("컴퓨트 워커 시작 실패: %v", err))
}
// 초기화 대기
<-computeWorker.Initialized()
fmt.Println("컴퓨트 워커 초기화 완료")
// 상태 모니터링
go func() {
for {
status := computeWorker.GetStatus()
fmt.Printf("워커 상태: %+v\n", status)
time.Sleep(10 * time.Second)
}
```

```
}()

// 종료 대기

<-computeWorker.Quit()

fmt.Println("컴퓨트 워커 종료")

}
```

문제 해결

일반적인 문제

1. **런타임 로딩 실패**

```
# 런타임 번들 확인

oasis-node bundle info runtime.orc

# 런타임 로더 테스트

oasis-core-runtime-loader --type sgxs --signature runtime.sig runtime.sgxs
```

2. **배치 처리 타임아웃**

```
# 설정에서 타임아웃 증가

runtime:

timeout: "30s"
```

3. **메모리 부족**

```
# 배치 크기 제한
```

```yaml
runtime:
max_batch_size: 100
max_batch_size_bytes: "1MB"
```

디버깅 도구

```bash
# 상세 로그 활성화
oasis-node --log.level debug --log.format json

# 특정 모듈 로그 필터링
oasis-node --log.level debug 2>&1 | grep "worker/executor"

# 메트릭 모니터링
curl -s http://localhost:9001/metrics | grep -E "(batch|runtime|storage)"

# 런타임 상태 덤프
oasis-node control runtime-stats --runtime-id <ID>
```

컴퓨트 워커는 Oasis Core의 트랜잭션 실행을 담당하는 핵심 컴포넌트로, 효율적인 배치 처리와 불일치 탐지를 통해 네트워크의 보안성과 성능을 보장합니다.